



Clinical Lab Analysis System

MIMIC-III Based Automated Clinical Laboratory Analysis

Research Question:

"How can laboratory test results during hospital admission be used to automatically answer a limited set of clinical analysis questions about hospitalized patients?"

Hala Hillou • Dana Nmarny • Anas Khoury • Alon Engel

Presentation Agenda

What we will cover today

- | | | | | | |
|----|-----------------------------------|--|----|---------------------------------|-----------------------------------|
| 01 | The Problem | Why manual MIMIC-III analysis fails | 11 | Advanced AI Mode | AgentCoder-style guarded code-gen |
| 02 | Our Solution | Modular, automated clinical lab system | 12 | User Interface | Streamlit dark-theme app |
| 03 | Dataset & Preparation | MIMIC-III: cleaning and merging | 13 | Live Demo: Trend Chart | Q3 hematocrit walkthrough |
| 04 | System Architecture | Registry-driven pipeline overview | 14 | Live Demo: NL Query | Same result via natural language |
| 05 | Example Workflow | End-to-end request walkthrough | 15 | Live Demo: Correction | Typo injection demo |
| 06 | 12 Supported Questions | Full question registry | 16 | Evaluation Results | 159 tests, 94% coverage |
| 07 | Code Generation | Template-based code builder | 17 | Sample Results | Real numbers from MIMIC-III |
| 08 | Execution & Validation | Safe exec + per-question rules | 18 | Engineering Quality | Design decisions & quality |
| 09 | Correction Loop | Bounded rule-based retry | 19 | Limitations & Future | Honest scope and roadmap |
| 10 | Natural Language Input | Controlled NL routing | 20 | Conclusion | Key takeaways |

1. The Problem

Why manual MIMIC-III analysis is not enough



Massive Dataset

LABELVENTS alone is ~2 GB with millions of records — impossible to query manually at scale



Error-Prone Queries

Writing SQL/Python per question is time-consuming, repetitive, and easy to get wrong



No Reproducibility

Ad-hoc queries can't be reliably repeated or audited by others



Clinical Urgency

Clinicians need answers quickly during hospital admissions — not hours of scripting

MIMIC-III at a Glance

~2 GB

LABELVENTS file size

1M+

Lab event rows (cleaned)

2,839

Unique admissions

2,234

Unique patients

376

Lab test types

2. Our Solution

A modular, automated clinical lab analysis system

For every predefined clinical question: generate executable Python code → run it → validate → auto-correct if needed → present the result with the code



Code Generation

Template-based Python code is generated from the selected question + parameters. Never free-text, always typed.



Safe Execution

Generated code runs in a controlled environment. Output is validated against expected columns and question-specific rules.



Auto-Correction

If the first run fails, a rule-based corrector fixes common errors and retries — with a hard attempt limit.



Transparent Results

Every answer ships with the exact generated code that produced it. Full traceability for clinical/research use.

3. Dataset & Data Preparation

MIMIC-III: ADMISSIONS, LABEVENTS, D_LABITEMS



ADMISSIONS

Patient admission records
HADM_ID, SUBJECT_ID
ADMITTIME, DISCHTIME, DIAGNOSIS



LABEVENTS

~2 GB of lab measurements
HADM_ID, ITEMID, CHARTTIME
VALUENUM, VALUEUOM, FLAG



D_LABITEMS

Lab item dictionary
ITEMID → LABEL
(376 readable test names)

Cleaning & Merging Steps

- 1 Schema inspection: confirmed required fields exist (HADM_ID, ITEMID, CHARTTIME, VALUENUM, VALUEUOM, FLAG, LABEL)
- 2 Filter: drop LABEVENTS rows with missing HADM_ID (required for the join)
- 3 Filter: keep only rows with valid VALUENUM for numeric analysis
- 4 Join: LABEVENTS → ADMISSIONS on HADM_ID to add admission context
- 5 Enrich: join D_LABITEMS (ITEMID → LABEL) to add readable test names

Result Dataset

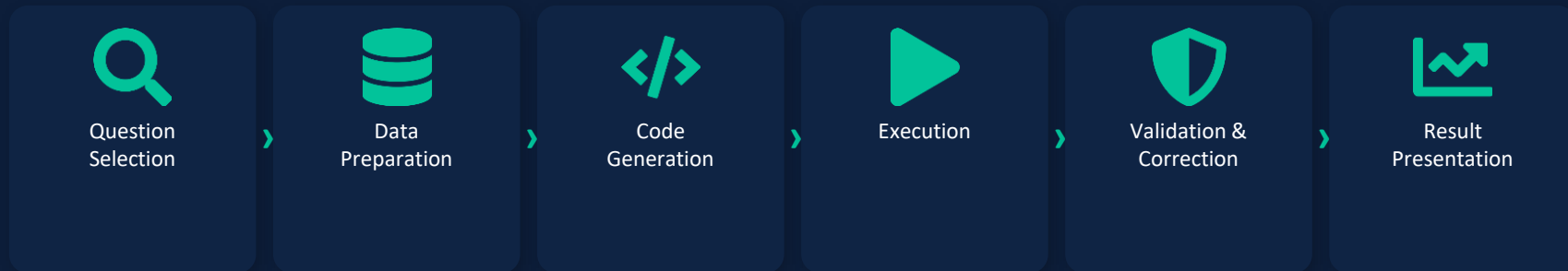
1,003,949

rows

12 columns · cleaned merged CSV
used by every analysis module

4. System Architecture

Registry-driven, layered pipeline



Single Source of Truth: all questions defined once in `questions.py` — UI, code generation, validation, and pipeline all read from it. Adding a question is a one-place change.

`questions.py`

Question registry – single source of truth

`data_prep.py`

Load dataset; one function per question

`code_generation.py`

Build runnable code string from spec + params

`execution.py`

Safe `exec()`; capture result/error/status

`validation.py`

Columns, shape, per-question rule checks

`correction.py`

Rule-based fixes + bounded retry loop

`question_router.py`

Controlled NL routing to template + params

`app.py`

Streamlit UI (dark theme)

`sandbox.py`

AST allowlist + restricted exec

`agent_pipeline.py`

Guarded LLM code-gen loop (experimental)

5. Example Workflow

End-to-end request: "Show the latest Glucose value for admission 145834"



User Request:

"Show the latest Glucose value for admission 145834"

1



Selects matching question template

Q2: 'Latest value of a lab test' → inputs: HADM_ID=145834, ITEMID=Glucose

2



Generates Python analysis code

```
result = get_latest_lab_value(145834, 50931)
```

3



Filters & processes the dataset

Filters ~1M rows → admission 145834 → Glucose → most recent CHARTTIME

4



Executes the code

exec() in controlled environment; captures result DataFrame + status

5



Validates the result

Checks: 1 row, expected columns, VALUENUM not null → Validation: Passed

6



Presents the clinical answer

Metric card: 'Latest value: 102 mg/dL' + Generated Code tab

6. The 12 Supported Questions

Base questions (Part 1: Q1–Q5) and new additions (Q6–Q12)

1 Abnormal lab results for an admission

Admission

7 Count abnormal vs normal results

Admission

2 Latest value of a selected lab test

Admission + Lab
Test

8 Abnormal results for a specific lab test

Admission + Lab
Test

3 Trend of a lab test over time

Admission + Lab
Test

9 List all distinct abnormal lab tests

Admission

4 All lab tests during an admission

Admission

10 Lab values within a date range

Admission + Lab
Test + Dates

5 Compare first vs last value of a lab test

Admission + Lab
Test

11 All admissions for a patient

Patient ID

6 Summary statistics (min / max / mean)

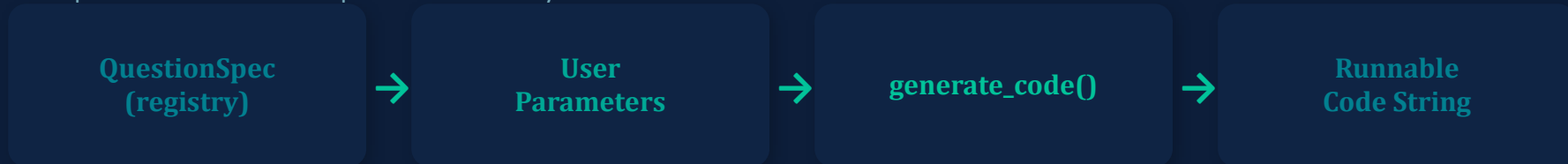
Admission + Lab
Test

12 Abnormal vs normal across ALL admissions

dataset-wide

7. Code Generation

Template-based: QuestionSpec → runnable Python



Generated Code — Q3: Lab Trend

```
# Generated by generate_code(spec, params)
result = get_lab_trend(hadm_id=107521,
itemid=51221)
```

```
# data function in data_prep.py:
def get_lab_trend(hadm_id, itemid):
    df = merged_df.copy()
    df = df[df['HADM_ID'] == hadm_id]
    df = df[df['ITEMID'] == itemid]
    return df.sort_values('CHARTTIME')
```



Only known functions

Generated code only ever calls a data function declared in data_prep.py — no free-text exec risk



Typed parameters

Admission IDs, ITEMIDs, and dates are validated before being injected into the code string



Every question → one template

Each QuestionSpec declares its function name; code generation is a one-line call per question



Full transparency

The generated code string is shown to the user in the 'Generated Code' tab

8. Execution & Validation

Safe exec() and per-question output checks



Execution (execution.py)

- ✓ Calls exec() on the generated code string
- ✓ Provides only the data functions namespace — no file/network access
- ✓ Captures: result DataFrame, error message, success status
- ✓ Returns structured ExecutionResult to the pipeline



Validation (validation.py)

- ✓ Result is a non-empty DataFrame
- ✓ Expected columns are all present
- ✓ Shape check (e.g. exactly 1 row for Q2 latest value)
- ✓ Per-question rules: time-ascending for trends, only FLAG=abnormal for Q1/Q8, counts add up for Q7/Q12

The screenshot shows the 'Validation Details' tab of the 'Clinical Lab Analysis System'. It displays a 'Selected Question' (Q1) and its 'Admission Context' (Admission ID: 014036_001, Patient ID: 014036_001, Date: 2024-04-01 08:00:00, Location: NETWORKS). The 'Selected Parameters' are: "trend": 1.0, "flag": 1.0. The 'Interpreted as' section shows: "[1] Show the trend of a selected lab test over time during an admission [admission: 014036_001, lab: Hemoglobin (Hb) (TEM01: 512.1)]." The 'Analysis completed successfully' message is displayed. The 'Execution' status is 'Execution completed successfully', 'Validation' is 'Validation passed', and 'Correction Applied' is 'No'. The 'Overall Status' is 'Execution and validation completed successfully'.

Validation Details tab

The screenshot shows the 'Clinical Lab Analysis System' interface. It displays a 'Selected Question' (Q2) and its 'Admission Context' (Admission ID: 014036_001, Patient ID: 014036_001, Date: 2024-04-01 08:00:00, Location: NETWORKS). The 'Selected Parameters' are: "trend": 1.0, "flag": 1.0. The 'Interpreted as' section shows: "[2] Return the latest value of a selected lab test during an admission [admission: 014036_001, lab: Hemoglobin (Hb) (TEM01: 512.1)]." The 'Analysis completed successfully' message is displayed. The 'Execution' status is 'Execution completed successfully', 'Validation' is 'Validation passed', and 'Correction Applied' is 'No'. The 'Overall Status' is 'Execution and validation completed successfully'.

Q2: Latest lab value — 1-row result

9. The Correction Loop

Rule-based retry when generated code fails

Generate Code



Execute



Validate



✓ Done

✗ Correct & Retry

↺ bounded retry (max N attempts)



How Correction Works



Whole-identifier matching

Regex matches only full function names (word boundaries) — won't corrupt a valid similar name like `get_abnormal_results_for_lab`



Known typo map (TYPO_FIXES)

A fixed map in `correction.py` covers the most common generated-code errors; `REVERSE_TYPOS` allows inverse repair



Bounded retry loop

Maximum attempt limit prevents infinite loops; if correction still fails, the error is surfaced cleanly to the user



Transparent attempts

Every attempt (code tried, error received, fix applied) is saved and shown in the 'Correction Attempts' tab

The screenshot displays the application's interface for handling a correction loop. At the top, there's a header with a 'Display' button. Below it, a section titled 'Admission ID (HADM_ID)' shows the value '140848'. A red 'Run Analysis' button is present. The main area is divided into two panels: 'Selected Question' and 'Admission Context'. The 'Selected Question' panel shows a question '[3] Show the trend of a selected lab test over time during an admission' and 'Selected Parameters' including 'Hadm_id' and 'Itemid'. The 'Admission Context' panel shows a table with columns 'HADM_ID', 'ADMITTIME', 'DISCHTIME', and 'DIAGNOSIS', containing data for admission 107521. Below these panels, a green bar indicates 'Interpreted as: [3] Show the trend of a selected lab test over time during an admission | admission 107521, lab itemid (ITEMID 51221)'. A green bar below that says 'Analysis completed successfully'. The bottom section shows 'Execution' (Execution completed successfully), 'Validation' (Passed), 'Correction Applied' (No), and 'Overall Status' (Execution and validation completed successfully). At the very bottom, a 'Correction Attempts' tab is visible, showing a table with columns 'HADM_ID', 'ITEMID', 'LABEL', 'VALUEIN', 'VALUEOUT', 'CHARTTIME', 'V234664', '107521-0', '51221-0', 'Hematocrit', '57.9', and '0'.

Correction Attempts tab — two attempts

10. Natural Language Input

Type a question in plain English — controlled routing to predefined templates

Clinical Lab Analysis System

MIMIC-III based system for predefined clinical laboratory questions

This system takes one predefined clinical laboratory question, generates executable Python code, runs it on the cleaned MIMIC-III laboratory dataset, validates the output, applies correction when needed, and presents the final result. It currently supports 11 question types, all defined in a single question registry.

✓ Ask a clinical lab question in natural language (optional)

Your question is routed to one of the supported question templates - no free-form code is generated or run. Examples:

- Show abnormal lab results for admission 145834
- What is the latest chloride value for admission 1998847
- Show hematocrit trend for admission 107521
- Compare first and last pO2 values for admission 177047

Your question

Show hematocrit trend for admission 107521

Interpret & Run

Question Selection

Category

All

Choose a clinical question

Show abnormal lab results for a specific admission

Lists every abnormal-flagged lab result for the admission. Inputs: admission.

Admission ID (HADM_ID)

145834.0

Run Analysis

Selected Question

[3] Show the trend of a selected lab test over time during an admission

Selected Parameters

```
{  "hadm_id": 107521  "item_id": 51221}
```

Admission Context

HADM_ID	ADMITTIME	DISCHTIME	DIAGNOSIS
107521	2158-04-05 08:54:00	2158-04-09 13:34:00	NEWBORN



Rules Only (default)

Regex / keyword matching maps free text to a template ID + params. Fast, offline, deterministic. Always available.



Rules → AI Fallback

If no rule matches and an LLM key is set (Anthropic / OpenAI / Gemini), the LLM selects the best template + params. Still never generates code.



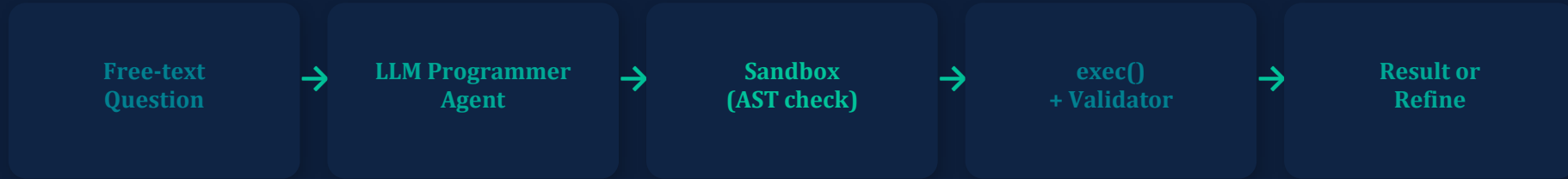
Advanced: AI Writes Code

Optional, off by default. LLM programmer agent writes a pandas snippet for free-form questions. Sandboxed + validated. See next slide.

Examples: "Show hematocrit trend for admission 107521" → Q3 | "What is the latest chloride for admission 199884?" → Q2 | "Show abnormal results for all admissions" → Q12

11. Advanced: AI Writes Code (Experimental)

AgentCoder-style guarded LLM code generation — off by default



Inspired by: AgentCoder (arXiv 2312.13010) — programmer agent + validator + executor, in a generate→test→refine loop. Orchestration is kept lightweight and in-repo (no CrewAI/AutoGen/LangGraph).



Sandbox Guardrails (sandbox.py)

- ✓ AST allowlist: no imports, no dunder/underscore attributes
- ✓ No file / network / eval / exec methods allowed
- ✓ No loops (for/while) in generated code
- ✓ Restricted `__builtins__`: only `pd` and a read-only `df` exposed
- ✓ Worker thread with soft timeout and row cap
- ✓ Result must pass `validate_freeform_result()` before display



When Does It Activate?

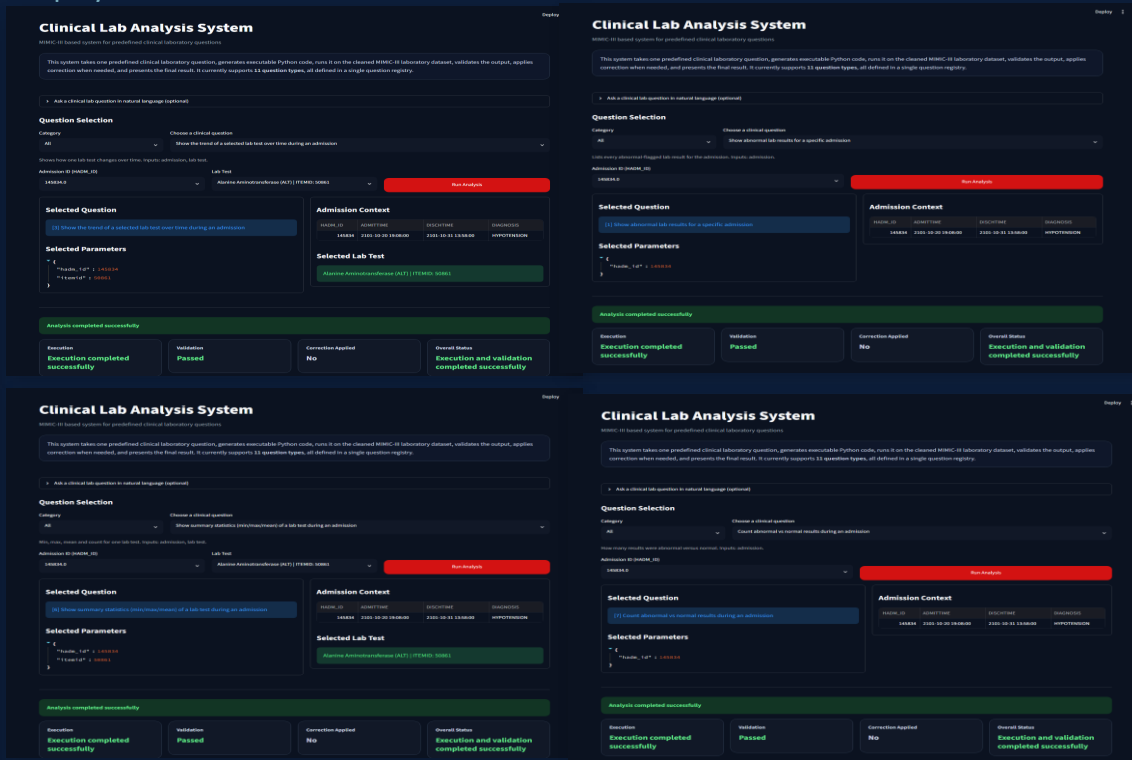
- ✓ **Off by default**
No LLM key set → Rules Only mode, always
- ✓ **Explicit selection**
User chooses 'Advanced: AI writes code' mode in the UI
- ✓ **Escalation dialog**
If template mode cannot match, a popup offers one-click switch to Advanced
- ✓ **Free-form questions**
Questions outside the 12-template registry (e.g. 'last 10 abnormal across the dataset')

12. User Interface (Streamlit)

Dark-themed app with dynamic inputs and rich result display

UI Features

- ✓ Question selector with category filter
- ✓ Per-question help caption (what it returns)
- ✓ Parameter inputs rendered dynamically from the registry
- ✓ Optional natural-language question box
- ✓ Summary metric cards (latest value, min/max/mean, counts)
- ✓ Altair trend line chart with axis labels + units
- ✓ Color-coded status: ✓ green / ✗ red / — blue no-records
- ✓ Generated Code tab: exact code that ran
- ✓ Validation Details tab
- ✓ Correction Attempts tab
- ✓ CSV download of results
- ✓ Pinned dark theme (config.toml)



Q3: Trend + chart Q6: Summary stats

Q1: Abnormal results Q7: Counts

13. Live Demo: Lab Trend Chart (Q3)

Hematocrit trend for admission 107521

What We Do

- ✓ Select Question: 'Show the trend of a selected lab test over time during an admission'
- ✓ Set Admission ID: 107521
- ✓ Set Lab Test: Hematocrit (ITEMID 51221)
- ✓ Click: Run Analysis
- ✓ Expected: line chart showing decreasing trend ~57.9% → 44.4%
- ✓ Generated Code tab: `result = get_lab_trend(107521, 51221)`
- ✓ Status: Execution ✓ Validation ✓ Correction: No

Clinical Lab Analysis System

MIMIC-III based system for predefined clinical laboratory questions

This system takes one predefined clinical laboratory question, generates executable Python code, runs it on the cleaned MIMIC-III laboratory dataset, validates the output, applies correction when needed, and presents the final result. It currently supports 11 question types, all defined in a single question registry.

Ask a clinical lab question in natural language (optional)

Question Selection

Category: All Choose a clinical question: Show the trend of a selected lab test over time during an admission

Shows how one lab test changes over time. Inputs: admission, lab test.

Admission ID (HADM_ID): 145834.0 Lab Test: Alanine Aminotransferase (ALT) | ITEMID: 50861 Run Analysis

Selected Question

[3] Show the trend of a selected lab test over time during an admission

Selected Parameters

```
{  "hadm_id": 145834,  "itemid": 50861}
```

Admission Context

HADM_ID	ADMITTIME	DISCHTIME	DIAGNOSIS
145834	2101-10-20 19:08:00	2101-10-31 13:58:00	HYPOTENSION

Selected Lab Test

Alanine Aminotransferase (ALT) | ITEMID: 50861

Analysis completed successfully

Execution
Execution completed successfully

Validation
Passed

Correction Applied
No

Overall Status
Execution and validation completed successfully

Result:

Hematocrit trend for admission 107521: decreasing from 57.9% → 44.4% over admission period. Line chart + data table + generated code shown.

14. Live Demo: Natural Language Query

Same result via plain English — no code needed from the user



User types: "Show hematocrit trend for admission 107521"

What the Router Does

- ✓ Keyword match: 'trend' → Q3 template
- ✓ Extract HADM_ID: 107521 from the sentence
- ✓ Match lab: 'hematocrit' → ITEMID 51221
- ✓ Build route result: question_id=3, params={...}
- ✓ Pass to pipeline — same path as dropdown
- ✓ Show: 'Interpreted as: [3] ... admission 107521, Hematocrit'



The router maps the sentence to a predefined template + parameters — it NEVER generates or executes free-form code. The result is identical to the dropdown path.

Clinical Lab Analysis System

MIMIC-III based system for predefined clinical laboratory questions

This system takes one predefined clinical laboratory question, generates executable Python code, runs it on the cleaned MIMIC-III laboratory dataset, validates the output, applies correction when needed, and presents the final result. It currently supports 11 question types, all defined in a single question registry.

▼ Ask a clinical lab question in natural language (optional)

Your question is routed to one of the supported question templates - no free-form code is generated or run. Examples:

- Show abnormal lab results for admission 145834
- What is the latest chloride value for admission 1998847
- Show hematocrit trend for admission 107521
- Compare first and last pO2 values for admission 177047

Your question

Show hematocrit trend for admission 107521

Interpret & Run

Question Selection

Category

All

Choose a clinical question

Show abnormal lab results for a specific admission

1 lab every abnormal flagged lab result for the admission. Inputs: admission.

Admission ID (HADM_ID)

140834.0

Selected Question

[3] Show the trend of a selected lab test over time during an admission

Selected Parameters

```
{
  "hadm_id": 107521,
  "itemid": 51221
}
```

Admission Context

HADM_ID	ADMITTIME	DISCHTIME	DIAGNOSIS
107521	2158-04-05 08:54:00	2158-04-09 13:34:00	NEWBORN

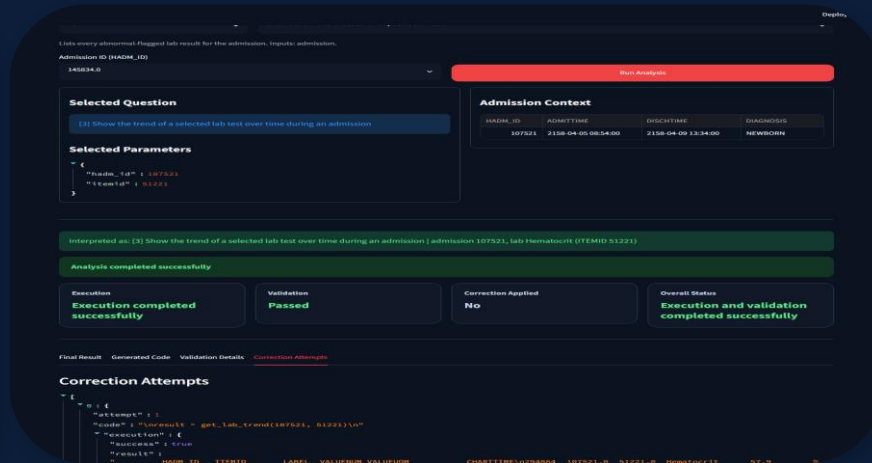
Run Analysis

15. Live Demo: Correction Loop

Injecting a typo to show auto-repair in action

How to Demo It

- ✓ Select any of Q1–Q5
- ✓ Check: 'Demo: inject a typo to force a correction'
- ✓ Click: Run Analysis
- ✓ Attempt 1 fails — function name is misspelled
- ✓ Corrector repairs the name → Attempt 2 succeeds
- ✓ Status row shows: Correction Applied = Yes
- ✓ Correction Attempts tab shows both attempts



Why This Matters

The template generator always emits correct code in normal use — the demo checkbox is the only reliable way to show the corrector firing live.

The corrector uses whole-identifier matching (word boundaries) to avoid corrupting valid names like `get_abnormal_results_for_lab`.

The bounded retry limit (N attempts) ensures the system never loops forever — if all attempts fail, the error is surfaced cleanly.

16. Evaluation Results

All 12 questions pass execution and validation

12 / 12

Questions Pass

159

Automated Tests

94%

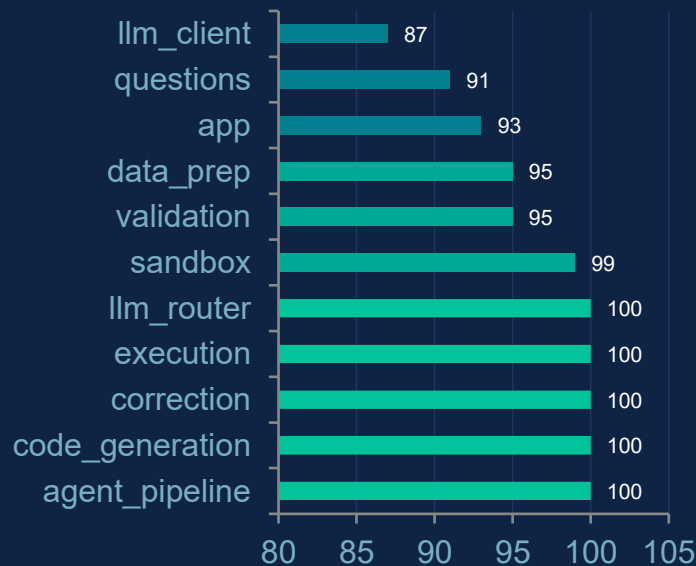
Line Coverage

85%

Coverage Gate

End-to-End Results (all from MIMIC-III dataset)

Q1	HADM 145834	Abnormal results returned (FLAG = abnormal)
Q2	HADM 199884, Chloride	Latest value: 103 mEq/L
Q3	HADM 107521, Hematocrit	Decreasing trend: 57.9% → 44.4%
Q4	HADM 158675	All lab tests listed
Q5	HADM 177047, pO2	First 82, Last 103, Diff +21 mm Hg
Q12	Dataset-wide	One row per admission, most abnormal first



17. Sample Results from MIMIC-III

Real output across multiple questions and admissions



Q2

Chloride Latest

103 mEq/L

HADM 199884



Q3

Hematocrit Trend

57.9% → 44.4%

HADM 107521 (decreasing)



Q5

pO2 First vs Last

+21 mm Hg

HADM 177047: 82 → 103

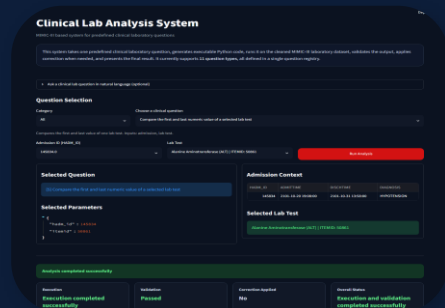


Q12

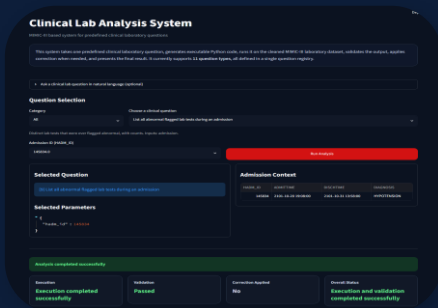
Dataset-Wide

2,839 admissions

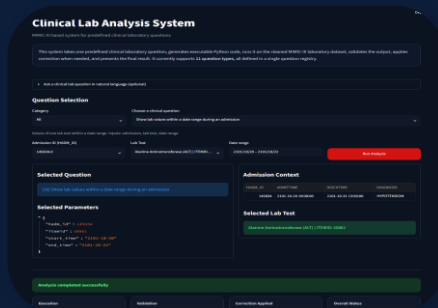
Abnormal / Normal counted per admission



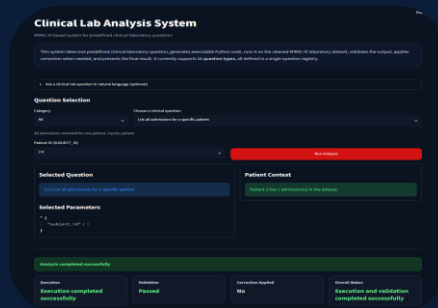
Q5: First vs Last (pO2)



Q9: Distinct abnormal tests



Q10: Date range values



Q11: Patient admissions

18. Engineering Quality & Design

Modular, tested, and maintainable



Single Source of Truth

All 12 questions in one registry (questions.py). Adding a question is a one-place change — no edits to app, pipeline, or validation.



Sandboxed Execution

AST allowlist + restricted builtins: no imports, no file/network, no loops. Worker thread timeout and row cap.



159 Tests, 94% Coverage

Unit tests for every module + end-to-end Streamlit AppTest for all 12 questions. 85% minimum coverage gate in pyproject.toml.



Optional AI, Controlled

LLM features are off by default. Both routing and code-gen use deterministic validators before any result is shown.



Template Safety

Generated code only calls declared data functions with typed parameters — never executes free-text or user-supplied code.



Secrets & Dependency Mgmt

API keys only in .env (gitignored). Dependencies managed with uv + pyproject.toml + committed uv.lock.

19. Limitations & Future Work

Honest scope and what comes next



Current Limitations

- Questions are predefined — free-form code-gen is experimental and off by default
- Single local CSV dataset — no live SQL backend or real-time hospital connection
- No authentication or multi-user state isolation
- UI and chart presentation polish deferred (future iteration)
- Empty results for valid-but-unmatched filters reported as 'no records' (not an error)



Future Work



Broaden the guarded code-gen catalog with more safe question types



Live SQL backend for real-time hospital data integration



Richer visualizations: heatmaps, multi-lab comparisons, distribution charts



Support for broader MIMIC-III tables: vital signs, clinical notes, medications



LLM-assisted question suggestion and parameter auto-fill

20. Conclusion & Key Takeaways

What we built and why it works



Automated Clinical Analysis

12 predefined questions answered reliably from MIMIC-III without manual scripting



Generated + Transparent Code

Every result ships with the exact Python code that produced it — full traceability



One Registry to Rule Them All

Single question registry drives UI, code gen, validation, and pipeline — zero duplication



Safe & Validated

Sandboxed execution, per-question validation, bounded correction, deterministic results



Rich User Experience

Metric cards, trend charts, NL input, dark theme, CSV export, generated-code tab



Production-Quality Engineering

159 tests, 94% coverage, secrets in .env, managed with uv, clean git history

Goal: Provide an automated, reliable, and user-friendly system for clinical laboratory data analysis.



Thank You

Clinical Lab Analysis System — MIMIC-III

Hala Hillou • Dana Nmarny • Anas Khoury • Alon Engel

12

Questions

159

Tests

94%

Tested Code Coverage

1M+

Lab Rows